

Retrieval Augmented Generation with Ollama and LangChain

ECE 595: AI Infrastructure

Scott Nguyen — snguyen9

Homework 4

1 Introduction

This assignment implements and evaluates a Retrieval Augmented Generation (RAG) system using a local large language model. The system processes a PDF by splitting it into chunks, embedding the text into a vector database, retrieving relevant context, and generating answers using an Ollama-hosted model.

The implementation uses LangChain utilities, Chroma for vector storage, and HuggingFace embeddings. Experiments were conducted in WSL, and results were recorded for two different RAG configurations.

2 System Setup

The system was implemented in Python using a virtual environment in WSL. Two configurations were tested using the following commands:

Listing 1: Configuration A run command

```
python run_questions.py \  
--chunk-size 1200 \  
--chunk-overlap 100 \  
--k 2 \  
--llm llama3.2:latest \  
--output results_config_a.txt
```

Listing 2: Configuration B run command

```
python run_questions.py \  
--chunk-size 500 \  
--chunk-overlap 50 \  
--k 4 \  
--llm llama3.2:latest \  
--output results_config_b.txt
```

3 RAG Pipeline Design

The system follows a standard RAG workflow:

1. Load the PDF using `PyPDFLoader`
2. Split text into chunks with `RecursiveCharacterTextSplitter`
3. Generate embeddings using `all-MiniLM-L6-v2`
4. Store embeddings in Chroma


```

Running question 3/8
Question 3: What is retrieval augmented generation?

Answer:
Retrieval Augmented Generation (RAG) is a technique that combines retrieval and generation capabilities to improve the performance of language models. It involves using an embedding model to convert documents and queries into vectors, which are then used to retrieve relevant context and generate responses.

Retrieved source pages:
Source 1: page 0
Source 2: page 5
Source 3: page 8
Source 4: page 2

```

Figure 6: Configuration B answering a RAG definition question.

7 Log Output

The automated test script saved full responses into log files.

7.1 Configuration A Log

Listing 3: Results from Configuration A

```

RAG Automated Test Results
=====

Chunk size: 1200
Chunk overlap: 100
Retriever k: 2
LLM: llama3.2:latest
Embedding model: all-MiniLM-L6-v2

Question 1: What is this homework about?

Answer:
This homework appears to be about improving and enhancing the performance of a conversational AI system, specifically the RAG (Reactive Agent for Generating) model. It involves demonstrating understanding of embedding model selection, implementing advanced techniques, and evaluating the system's performance using various methodologies.

Retrieved source pages:
Source 1: page 8
Source 2: page 7

-----

Question 2: What are the main steps in building the RAG system?

Answer:
I do not know based on the document. The document does not explicitly outline the main steps in building the RAG system, but rather provides guidelines for improving its performance, implementing advanced techniques, and evaluating its performance using various methodologies.

Retrieved source pages:
Source 1: page 7
Source 2: page 6

-----

```

Question 3: What is retrieval augmented generation?

Answer:

Retrieval Augmented Generation (RAG) refers to a type of conversational AI system that combines the strengths of both retrieval and generation models. In RAG, a retrieval model is used to retrieve relevant information from a large dataset or vector store, which is then used as input to a generation model to generate a response.

Retrieved source pages:

Source 1: page 0

Source 2: page 7

Question 4: What embedding model is suggested in the homework?

Answer:

The suggested embedding model is "all-mpnet-base-v2" from the sentence-transformers library.

Retrieved source pages:

Source 1: page 2

Source 2: page 3

Question 5: What vector database is used?

Answer:

The vector store used for the RAG system is Chroma, but it's also mentioned that you're free to use any vector store of your choice.

Retrieved source pages:

Source 1: page 0

Source 2: page 8

Question 6: What are two ways to improve the RAG system?

Answer:

According to Source 1, Page 7, there are two ways to improve the RAG system:

1. Implementing and evaluating at least two different techniques to improve the RAG system's performance.
2. Comparing performance with different configurations.

Retrieved source pages:

Source 1: page 7

Source 2: page 6

Question 7: What percentage is assigned to performance improvement?

Answer:

According to Source 1 | Page 7, 15% is assigned to improving the RAG system's performance.

Retrieved source pages:

Source 1: page 7

Source 2: page 7

Question 8: What is the capital of France?

Answer:

I do not know based on the document. The question "What is the capital of France?" does not appear in the provided conversation history or document context.

Retrieved source pages:

Source 1: page 8

Source 2: page 2

7.2 Configuration B Log

Listing 4: Results from Configuration B

RAG Automated Test Results

=====

Chunk size: 500

Chunk overlap: 50

Retriever k: 4

LLM: llama3.2:latest

Embedding model: all-MiniLM-L6-v2

Question 1: What is this homework about?

Answer:

This homework is about building a Retrieval Augmented Generation (RAG) Application using Ollama and Langchain, with the goal of improving the performance of the model and optimizing it.

Retrieved source pages:

Source 1: page 7

Source 2: page 0

Source 3: page 8

Source 4: page 7

Question 2: What are the main steps in building the RAG system?

Answer:

Based on the document context, I do not know based on the document. The document provides information on evaluation methodology, improvement techniques, and core RAG implementation, but it does not explicitly outline the main steps in building the RAG system.

Retrieved source pages:

Source 1: page 7
Source 2: page 8
Source 3: page 6
Source 4: page 3

Question 3: What is retrieval augmented generation?

Answer:

Retrieval Augmented Generation (RAG) is a technique that combines retrieval and generation capabilities to improve the performance of language models. It involves using an embedding model to convert documents and queries into vectors, which are then used to retrieve relevant context and generate responses.

Retrieved source pages:

Source 1: page 0
Source 2: page 5
Source 3: page 8
Source 4: page 2

Question 4: What embedding model is suggested in the homework?

Answer:

The document suggests using HuggingFaceEmbeddings, which can be installed via pip: `!pip install sentence-transformers`.

Retrieved source pages:

Source 1: page 7
Source 2: page 6
Source 3: page 3
Source 4: page 2

Question 5: What vector database is used?

Answer:

The document suggests using Chroma as the vector store.

Retrieved source pages:

Source 1: page 0
Source 2: page 0
Source 3: page 6
Source 4: page 4

Question 6: What are two ways to improve the RAG system?

Answer:

According to Source 1 | Page 7, two ways to improve the RAG system are:

1. Implementing and evaluating at least two different techniques to improve the RAG system's performance.
2. Prompt engineering variations.

Retrieved source pages:

Source 1: page 7
Source 2: page 6
Source 3: page 0
Source 4: page 2

Question 7: What percentage is assigned to performance improvement?

Answer:

According to the document context, 30% of the grading rubric is assigned to Performance Evaluation and Improvement (Critical Thinking).

Retrieved source pages:

Source 1: page 7
Source 2: page 7
Source 3: page 7
Source 4: page 6

Question 8: What is the capital of France?

Answer:

I do not know based on the document. The question about vector database used does not have an answer in the provided conversation history and document context.

Retrieved source pages:

Source 1: page 7
Source 2: page 0
Source 3: page 4
Source 4: page 8

8 Performance Evaluation

The system was tested using eight questions covering general understanding, factual retrieval, improvement suggestions, rubric lookup, and system limitations. A negative test, “What is the capital of France?”, was included to verify that the model restricts answers to document context.

Correct behavior was to respond with “I do not know based on the document,” confirming proper grounding.

Table 2: Evaluation Question Categories

Question Type	Purpose
General summary	Tests document-level understanding
RAG definition	Tests conceptual explanation
Embedding model	Tests specific retrieval
Vector database	Tests Chroma retrieval
Improvement question	Tests rubric understanding
Negative test	Tests refusal for out-of-document questions

9 Analysis of Results

Configuration B generally produced more accurate responses due to increased context retrieval. For example, it better identified the homework as building a RAG application and correctly retrieved Chroma as the vector store. Configuration A was faster but sometimes missed relevant information because it retrieved fewer chunks.

Some inconsistencies were observed in both configurations, especially when the retrieved context was incomplete. This shows that RAG quality depends on chunking, retriever settings, prompt design, and model capability.

10 Design Decisions

The embedding model `all-MiniLM-L6-v2` was selected for efficiency in a CPU-based WSL environment. The model `llama3.2:latest` provided a balance between speed and answer quality.

Chroma persistence was used to avoid recomputing embeddings on every run. The automated runner was created so the same questions could be tested consistently across configurations.

11 Limitations

The system has several limitations. Retrieval quality depends on chunk size, overlap, and embedding accuracy. Smaller local models may also produce incomplete or vague responses.

The system does not use re-ranking, hybrid keyword search, or advanced document filtering. It also relies on static chunking, so it may not always retrieve the most useful context for every question.

12 Future Improvements

Future work could improve the system by:

- Testing stronger local models such as Mistral
- Adding a re-ranking step after vector retrieval
- Improving the prompt with more structured output requirements
- Adding hybrid retrieval using both keyword and vector search
- Showing direct source snippets with each answer

13 User Guide

Run the chatbot interactively:

```
python rag_chatbot.py --chunk-size 500 --chunk-overlap 50 --k 4 --llm llama3  
.2:latest
```

Run automated evaluation:

```
python run_questions.py --chunk-size 500 --chunk-overlap 50 --k 4 --llm llama3  
.2:latest --output logs/results_config_b.txt
```

The chatbot retrieves context, generates an answer, prints source pages, and can save results to a log file.

14 Conclusion

This project demonstrates how RAG improves answer quality by grounding responses in document context. The system successfully loaded a PDF, created embeddings, stored them in Chroma, retrieved relevant chunks, and generated answers with a local Ollama model.

Smaller chunks with higher retrieval values improved accuracy, while larger chunks provided faster execution. Overall, the project showed that RAG performance depends on chunking strategy, retrieval size, prompt design, and model selection.

A Appendix A: RAG Chatbot Source Code

Listing 5: rag_chatbot.py

```
import os
import argparse
from tqdm import tqdm

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader
from langchain_community.vectorstores import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.llms import Ollama

PDF_PATH = "data/ece-595-hw-3.pdf"

def build_vector_db(pdf_path, chunk_size, chunk_overlap, embedding_model):
    persist_dir = f"chroma_db_chunk{chunk_size}_overlap{chunk_overlap}"

    print("Loading embedding model...")
    embeddings = HuggingFaceEmbeddings(model_name=embedding_model)

    if os.path.exists(persist_dir):
        print(f"Loading existing vector DB: {persist_dir}")
        return Chroma(
            persist_directory=persist_dir,
            embedding_function=embeddings,
        )

    if not os.path.exists(pdf_path):
        raise FileNotFoundError(f"Could not find PDF at: {pdf_path}")

    print("Loading PDF...")
    pages = PyPDFLoader(pdf_path).load()
    print(f"Loaded {len(pages)} PDF pages.")

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
    )

    print("Splitting pages into chunks...")
    documents = []
    for page in tqdm(pages, desc="Chunking PDF"):
        documents.extend(splitter.split_documents([page]))

    print(f"Total chunks: {len(documents)}")

    print(f"Creating new vector DB: {persist_dir}")
    vectordb = Chroma.from_documents(
        documents=documents,
        embedding=embeddings,
        persist_directory=persist_dir,
    )

    return vectordb

def answer_question(llm, retriever, question, chat_history):
    docs = retriever.invoke(question)

    context = "\n\n".join(
        f"[Source {i + 1} | Page {doc.metadata.get('page', 'unknown')}] \n{doc.page_content}"
        for i, doc in enumerate(docs)
    )

    history_text = "\n".join(
        f"User: {q}\nAssistant: {a}" for q, a in chat_history[-3:]
    )
```

```

    )

    prompt = f"""
You are a precise RAG chatbot.

Use ONLY the document context to answer the question.
If the answer is not clearly in the document context, say:
"I do not know based on the document."

Keep the answer clear and concise.

Conversation history:
{history_text}

Document context:
{context}

Question:
{question}

Answer:
"""

    answer = llm.invoke(prompt)
    return answer, docs

def create_rag(args):
    vectordb = build_vector_db(
        pdf_path=args.pdf,
        chunk_size=args.chunk_size,
        chunk_overlap=args.chunk_overlap,
        embedding_model=args.embedding_model,
    )

    retriever = vectordb.as_retriever(search_kwargs={"k": args.k})
    llm = Ollama(model=args.llm, num_predict=80, temperature=0.2)

    return llm, retriever

def main():
    parser = argparse.ArgumentParser(description="RAG chatbot for ECE 595 HW3")

    parser.add_argument("--pdf", default=PDF_PATH)
    parser.add_argument("--chunk-size", type=int, default=1000)
    parser.add_argument("--chunk-overlap", type=int, default=150)
    parser.add_argument("--k", type=int, default=4)
    parser.add_argument("--llm", default="llama3.2:1b")
    parser.add_argument("--embedding-model", default="all-MiniLM-L6-v2")

    args = parser.parse_args()

    print("RAG Configuration")
    print("-----")
    print(f"PDF: {args.pdf}")
    print(f"Chunk size: {args.chunk_size}")
    print(f"Chunk overlap: {args.chunk_overlap}")
    print(f"Retriever k: {args.k}")
    print(f"LLM: {args.llm}")
    print(f"Embedding model: {args.embedding_model}")
    print()

    llm, retriever = create_rag(args)
    chat_history = []

    print("\nRAG chatbot is ready.")
    print("Ask questions about the PDF.")
    print("Type 'exit' to quit.\n")

    while True:

```

```
question = input("Question: ").strip()

if question.lower() in ["exit", "quit"]:
    print("Goodbye.")
    break

if not question:
    print("Please enter a question.\n")
    continue

answer, docs = answer_question(llm, retriever, question, chat_history)
chat_history.append((question, answer))

print("\nAnswer:")
print(answer)

print("\nRetrieved source pages:")
for i, doc in enumerate(docs, start=1):
    print(f"  Source {i}: page {doc.metadata.get('page', 'unknown')}")
print()

if __name__ == "__main__":
    main()
```

B Appendix B: Automated Question Runner Source Code

Listing 6: run_questions.py

```
import argparse
from rag_chatbot import create_rag, answer_question

QUESTIONS = [
    "What is this homework about?",
    "What are the main steps in building the RAG system?",
    "What is retrieval augmented generation?",
    "What embedding model is suggested in the homework?",
    "What vector database is used?",
    "What are two ways to improve the RAG system?",
    "What percentage is assigned to performance improvement?",
    "What is the capital of France?",
]

def main():
    parser = argparse.ArgumentParser(description="Automated RAG question runner")

    parser.add_argument("--pdf", default="data/ece-595-hw-3.pdf")
    parser.add_argument("--chunk-size", type=int, default=1000)
    parser.add_argument("--chunk-overlap", type=int, default=150)
    parser.add_argument("--k", type=int, default=4)
    parser.add_argument("--llm", default="mistral")
    parser.add_argument("--embedding-model", default="all-MiniLM-L6-v2")
    parser.add_argument("--output", default="rag_results.txt")

    args = parser.parse_args()

    llm, retriever = create_rag(args)
    chat_history = []

    with open(args.output, "w", encoding="utf-8") as f:
        header = f"""
RAG Automated Test Results
=====

Chunk size: {args.chunk_size}
Chunk overlap: {args.chunk_overlap}
Retriever k: {args.k}
LLM: {args.llm}
Embedding model: {args.embedding_model}

"""
        print(header)
        f.write(header)

        for i, question in enumerate(QUESTIONS, start=1):
            print(f"Running question {i}/{len(QUESTIONS)}")
            print(f"Question {i}: {question}")

            answer, docs = answer_question(llm, retriever, question, chat_history)
            chat_history.append((question, answer))

            console_output = f"""
Answer:
{answer.strip()}

Retrieved source pages:
"""
            print(console_output)
            f.write(f"Question {i}: {question}\n")
            f.write(console_output)

            for j, doc in enumerate(docs, start=1):
                page = doc.metadata.get("page", "unknown")
```

```
        source_line = f"  Source {j}: page {page}"
        print(source_line)
        f.write(source_line + "\n")

        separator = "\n" + "-" * 60 + "\n"
        print(separator)
        f.write(separator + "\n")

    print(f"Done. Results saved to {args.output}")

if __name__ == "__main__":
    main()
```